GraphCompose feature catalog

Each block below is self-documenting: the heading lands in the PDF outline (use your viewer's bookmarks panel as a clickable index), the grey panel shows the exact API call, and the result renders right under it. Blocks never split across pages — that's keepTogether(), also catalogued here.

Rich text — mixed runs in one paragraph

```
section.addRich(r -> r
    .plain("Status: ").bold("Approved ").checkbox(9, true, TEAL)
    .plain("  budget ").accent("$1.2M", GOLD)
    .plain("  details: ").link("docs", "https://github.com/DemchaAV/GraphCompose"))
```

Status: **Approved** ☑ budget **$1.2M** details: docs

Inline sparklines — mini-charts on the text baseline

```
section.addRich(r -> r
    .plain("Revenue trend ").sparkline(42, 9, TEAL, 65.2, 69.8, 74.1, 81.3, 88.2)
    .plain("   profit ").sparklineLine(42, 9, 1.6, GOLD, 28.1, 30.7, 32.9, 36.4, 39.5))
```

Revenue trend  profit 

Nested lists — per-depth markers

```
section.addList(l -> l
    .markerFor(1, ListMarker.custom("–")).markerFor(2, ListMarker.custom("·"))
    .addItem("Platform", nested -> nested
        .addItem("Layout engine")
        .addItem("Backends", deep -> deep.addItem("PDF").addItem("DOCX")))
    .addItem("Tooling"))
```

- Platform
  - Layout engine
  - Backends
    - PDF
    - DOCX
- Tooling

Timeline — markers on a connector rail

```
section.addTimeline(t -> t.keepEntriesTogether()
    .entry(TimelineMarker.numbered(1, 14, TEAL, WHITE),
           e -> e.title("Kickoff").meta("Jan 2026").body("Scope agreed."))
    .entry(TimelineMarker.dot(8, GOLD),
           e -> e.title("Beta").meta("Mar 2026").body("First external users.")))
```

1 **Kickoff**
Jan 2026
Scope agreed.

Beta
Mar 2026
First external users.

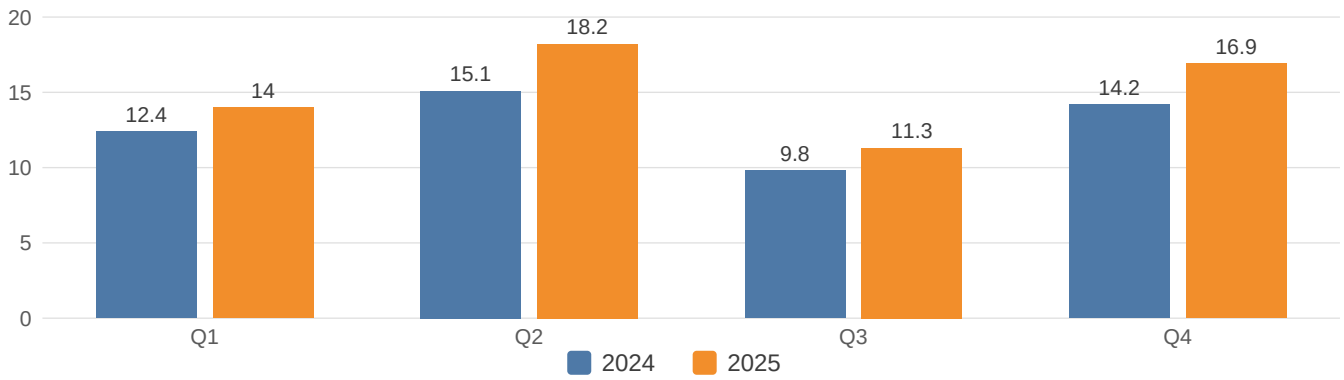
Tables — zebra rows and a totals row

```
section.addTable(t -> t
    .columns(DocumentTableColumn.auto(), DocumentTableColumn.auto(), DocumentTableColumn.auto())
    .headerRow("Region", "Q1", "Q2")
    .row("EMEA", "38", "41").row("APAC", "22", "25")
    .zebra(WHITE, rgb(248, 246, 240))
    .totalRow(totalStyle, "Total", "60", "66"))
```

Region	Q1	Q2
EMEA	38	41
APAC	22	25
Total	60	66

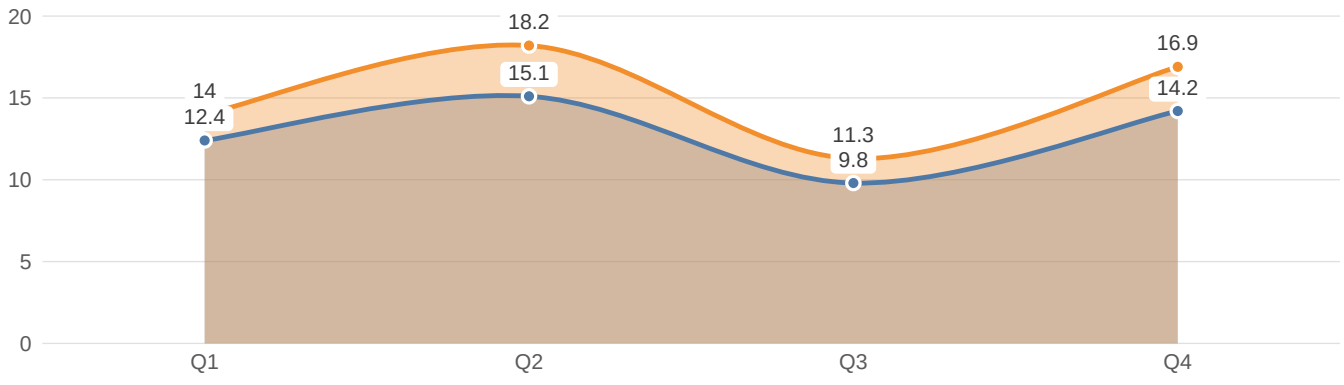
Bar chart — grouped, value labels, legend

```
section.chart(ChartSpec.bar().data(revenue)
    .valueLabels(ValueLabelMode.OUTSIDE)
    .legend(LegendPosition.BOTTOM)
    .size(ChartSize.fixedHeight(150)).build())
```



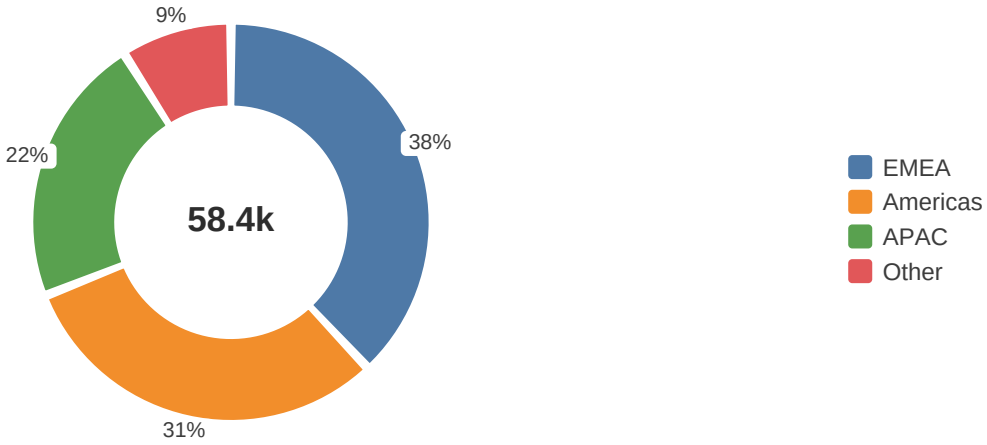
Line chart — smooth, area fill, markers, label halos

```
section.chart(ChartSpec.line().data(revenue)
        .interpolation(LineInterpolation.SMOOTH).area(true).valueLabels(ValueLabelMode.OUTSIDE)
        .size(ChartSize.fixedHeight(150)).build(),
    ChartStyle.builder().lineWidth(1.8)
        .pointMarker(PointMarker.circle(5).withStroke(DocumentStroke.of(WHITE, 1.2)))
        .build())
```



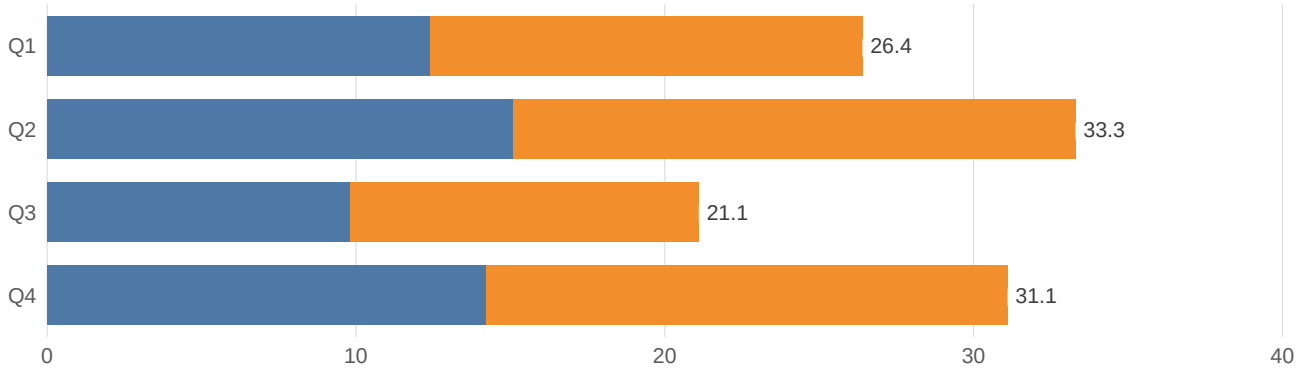
Donut chart — centre KPI, slice gaps, percent labels

```
section.chart(ChartSpec.pie().data(regions)
        .donutRatio(0.58).centerText("58.4k")
        .sliceLabels(SliceLabelMode.PERCENT)
        .legend(LegendPosition.RIGHT)
        .size(ChartSize.fixedHeight(170)).build(),
    ChartStyle.builder().sliceGapDegrees(2).build())
```



Horizontal bars — categories on Y, stacked totals

```
section.chart(ChartSpec.bar().data(revenue)
    .horizontal(true).grouping(BarGrouping.STACKED)
    .valueLabels(ValueLabelMode.OUTSIDE)
    .size(ChartSize.fixedHeight(140)).build())
```



Images — classpath bytes, explicit size, fit modes

```
DocumentImageData photo = DocumentImageData.fromBytes(
    getClass().getResourceAsStream("/engine-hero.jpg").readAllBytes());
section.addImage(i -> i.source(photo).size(150, 84)
    .fitMode(DocumentImageFitMode.COVER))   // vs CONTAIN on the right
```

Shapes — dividers, ellipses, soft cards

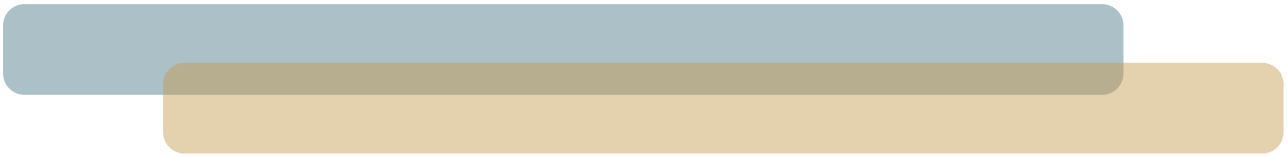
```
section.addDivider(d -> d.width(420).thickness(2).color(GOLD));
section.addEllipse(96, 44, TEAL);
section.addShape(s -> s.size(150, 40).cornerRadius(10)
    .fillColor(rgb(248, 246, 240)).stroke(DocumentStroke.of(GOLD, 0.8)))
```

Gradient fills — native linear and radial shadings

```
section.addShape(s -> s.size(420, 40).cornerRadius(8)
    .fill(DocumentPaint.linear(TEAL, GOLD)));            // 0 deg = left -> right
section.addShape(s -> s.size(420, 40).cornerRadius(8)
    .fill(new DocumentPaint.Radial(stops(GOLD, NAVY), 0.5, 0.5)))
```

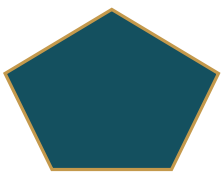
Translucency — rgba colours blend on overlap

```
section.addShape(s -> s.size(420, 34).cornerRadius(8)
    .fillColor(TEAL.withOpacity(0.35)));
// alpha reaches the PDF as a graphics-state constant;
// opaque colours stay byte-identical
```



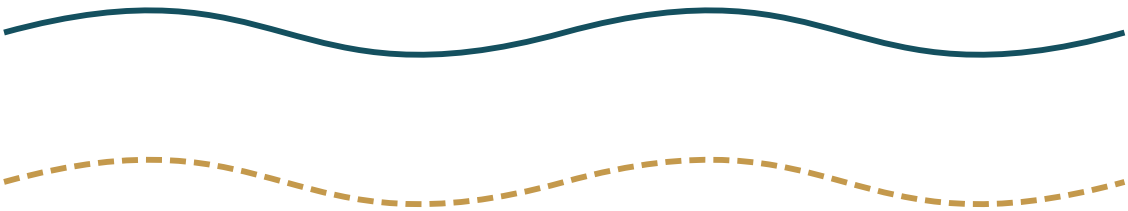
Custom polygons — arbitrary closed vector geometry

```
section.add(new PolygonNode("Gem", 80, 60,
    List.of(new ShapePoint(0.5, 1), new ShapePoint(1, 0.6),
            new ShapePoint(0.78, 0), new ShapePoint(0.22, 0),
            new ShapePoint(0, 0.6)),
    TEAL, DocumentStroke.of(GOLD, 1.2), zero(), zero()))
```



Vector paths — native Bézier curves, dashed strokes

```
section.addPath(path -> path.size(420, 48)   // unit box, y-up; controls may overshoot
    .moveTo(0.0, 0.5)
    .curveTo(0.25, 1.1, 0.25, -0.1, 0.5, 0.5)
    .curveTo(0.75, 1.1, 0.75, -0.1, 1.0, 0.5)
    .stroke(DocumentStroke.of(TEAL, 2.2)));
// .dashed(6, 3) turns the same stroke into a dash pattern that follows the curve
```



SVG path import (beta) — any d string as native curves

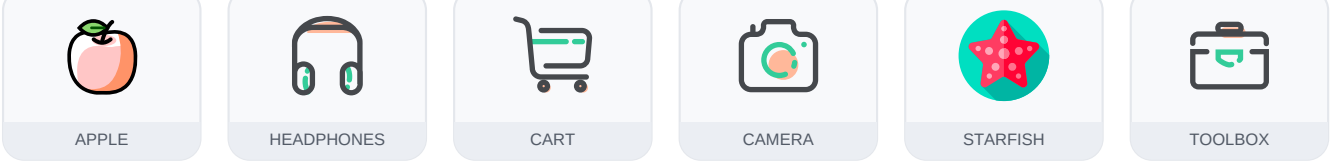
```
// Material Icons "favorite" (Apache 2.0), viewBox 0 0 24 24
section.addPath(path -> path.size(64, 64)
    .svg(SvgPath.parse(HEART_D, 0, 0, 24, 24))   // curves stay native PDF operators
    .fillColor(rgb(196, 30, 58)))
```

SVG-path clip — content clipped to a curve silhouette (CLIP_PATH)

```
section.addContainer(card -> card
    .path(72, 72, SvgPath.parse(HEART_D, 0, 0, 24, 24))
    .clipPolicy(ClipPolicy.CLIP_PATH)          // full-box gradient renders heart-shaped
    .layer(new ShapeBuilder().size(72, 72)
        .fill(DocumentPaint.linear(GOLD, TEAL)).build()))
```

SVG icons (beta) — multicolour files centred on tile cards

```
SvgIcon icon = SvgIcon.read(Path.of("icons/apple.svg"));  // layers + resolved paints
card.roundedRect(74, 64, 8)                              // fixed box = the tile
    .position(icon.node(34), 0, -7, LayerAlign.CENTER)  // icon.node keeps its tight
    .bottomCenter(plaque("APPLE"))                       // box, so CENTER lands true
```

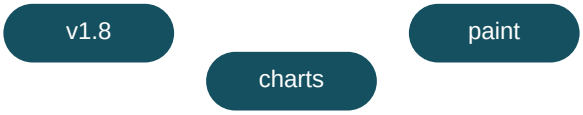


Shape as container — children clipped to the outline

```
section.addCircle(72, TEAL, c -> c
    .stroke(DocumentStroke.of(GOLD, 1.2))
    .center(label("GC", 20, WHITE)))
```

Canvas — absolute (x, y) placement

```
section.addCanvas(220, 70, canvas -> canvas
    .position(badge("v1.8"), 8, 8)
    .position(badge("charts"), 84, 26)
    .position(badge("paint"), 160, 8))
```



Transforms — render-time rotation

```
section.addShape(s -> s.size(120, 26).cornerRadius(13)
    .fillColor(GOLD).rotate(-6))
```

Barcodes — QR and Code 128 with theme tinting

```
section.addBarcode(b -> b.qrCode()
    .data("https://github.com/DemchaAV/GraphCompose")
    .foreground(TEAL).size(86, 86));
section.addBarcode(b -> b.code128().data("GC-2026-001").size(200, 44))
```

Debug overlay — corner badges name every box

```
GraphCompose.document(out)
    .debug(DocumentDebugOptions.nodeLabels())   // or guidesAndNodeLabels()
    .create();
// LabelText.FULL_PATH prints whole layout paths; guides() adds box outlines
```

Re-render any document with nodeLabels() and every placed box grows a corner badge with its semantic name (PriceSummaryTitle[0], SvgLayer3, …) — a misplaced component traces straight back to the builder call that produced it. The full annotated sheet is debug-overlay.pdf among the examples.

Page chrome — this document's own header, footer, outline

```
document.metadata(DocumentMetadata.builder().title("…").author("…").build());
document.header(DocumentHeaderFooter.builder().zone(HEADER)
    .leftText("GraphCompose · Feature catalog").rightText("{date}")…);
document.footer(…centerText("Page {page} of {pages}")…);
paragraph.bookmark(new DocumentBookmarkOptions("Feature catalog", 0))
```

Look around: the running header and the page-X-of-Y footer on every page come from the calls above, and every block heading is a clickable entry in the PDF bookmarks panel. A document.watermark(…) call stamps text or an image behind or above the content the same way.